

1.Topic Name

Student Record Management Using Singly Linked List

2. Introduction

A singly linked list is a dynamic data structure in which each node contains data and a pointer to the next node. In this project, a linked list is used to store student records such as Roll Number, Name, and Marks.

The program performs four main operations:

Insertion – Add a new student record.

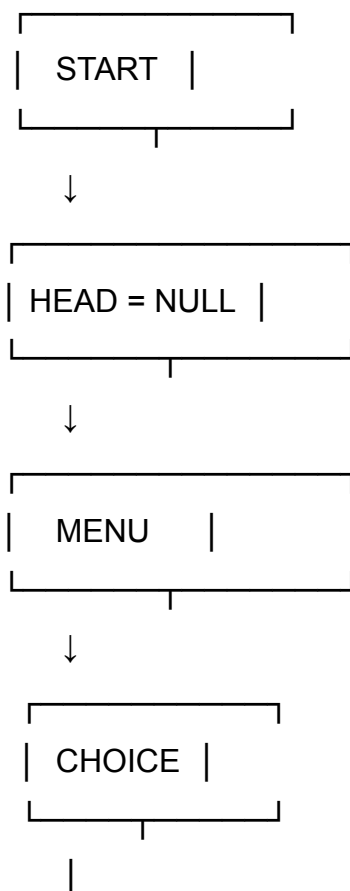
Deletion – Delete a student record using roll number.

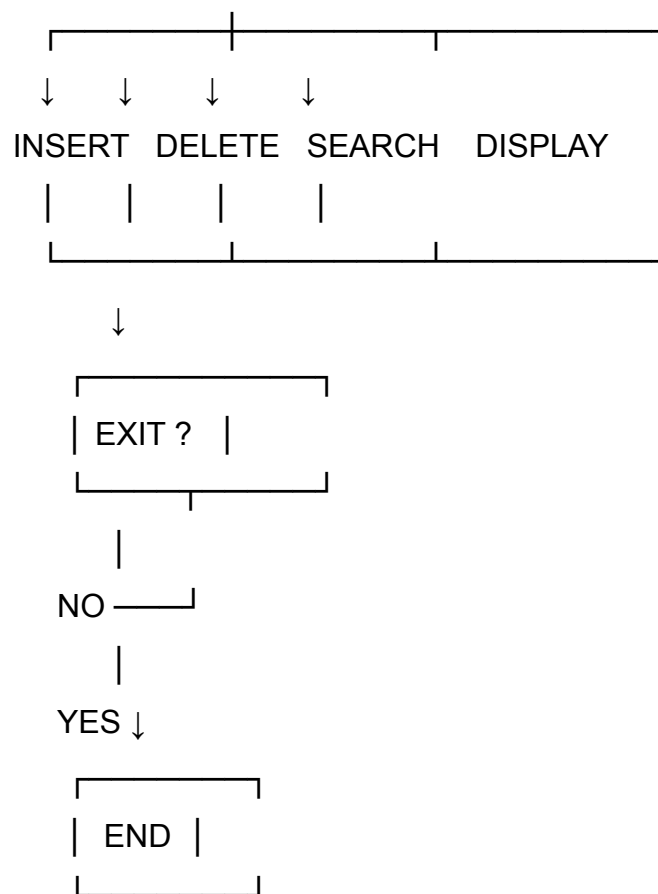
Searching – Search for a student using roll number.

Display – Display all student records.

This project demonstrates the practical use of linked lists in Student Record Management.

3. Flow Diagram





4.C++ Program

```
#include <iostream>
```

```
using namespace std;
```

```
struct Node
```

```
{
    int roll;
    string name;
    float marks;
    Node* next;
};
```

```
Node* head = NULL;
```

```
// Insert student
```

```
void insert()
```

```
{  
    Node* newNode = new Node;  
  
    cout << "Enter Roll No: ";  
    cin >> newNode->roll;  
  
    cout << "Enter Name: ";  
    cin >> newNode->name;  
  
    cout << "Enter Marks: ";  
    cin >> newNode->marks;  
  
    newNode->next = head;  
    head = newNode;  
  
    cout << "Record inserted successfully.\n";  
}
```

// Delete student

```
void deleteRecord()  
{  
    int roll;  
    cout << "Enter Roll No to delete: ";  
    cin >> roll;  
  
    Node *temp = head, *prev = NULL;  
  
    while (temp != NULL && temp->roll != roll)  
    {  
        prev = temp;  
        temp = temp->next;  
    }
```

```
if (temp == NULL)
{
    cout << "Record not found.\n";
    return;
}
```

```
if (prev == NULL)
    head = temp->next;
else
    prev->next = temp->next;
```

```
delete temp;
cout << "Record deleted successfully.\n";
}
```

// Search student

```
void search()
{
    int roll;
    cout << "Enter Roll No to search: ";
    cin >> roll;

    Node* temp = head;

    while (temp != NULL)
    {
        if (temp->roll == roll)
        {
            cout << "Record Found!\n";
            cout << "Roll No: " << temp->roll << endl;
            cout << "Name: " << temp->name << endl;
            cout << "Marks: " << temp->marks << endl;
            return;
        }
    }
}
```

```
}
```

```
temp = temp->next;
```

```
}
```

```
cout << "Record not found.\n";
```

```
}
```

```
// Display students
```

```
void display()
```

```
{
```

```
Node* temp = head;
```

```
if (temp == NULL)
```

```
{
```

```
cout << "No records available.\n";
```

```
return;
```

```
}
```

```
cout << "\nStudent Records:\n";
```

```
while (temp != NULL)
```

```
{
```

```
cout << "Roll No: " << temp->roll << endl;
```

```
cout << "Name: " << temp->name << endl;
```

```
cout << "Marks: " << temp->marks << endl;
```

```
cout << "-----\n";
```

```
temp = temp->next;
```

```
}
```

```
}
```

```
int main()
```

```

{
    int choice;

    do
    {
        cout << "\n===== Student Record Management =====\n";
        cout << "1. Insert\n";
        cout << "2. Delete\n";
        cout << "3. Search\n";
        cout << "4. Display\n";
        cout << "5. Exit\n";

        cout << "Enter choice: ";
        cin >> choice;

        switch (choice)
        {
            case 1: insert(); break;
            case 2: deleteRecord(); break;
            case 3: search(); break;
            case 4: display(); break;
            case 5: cout << "Program ended.\n"; break;
            default: cout << "Invalid choice.\n";
        }

    } while (choice != 5);

    return 0;
}

```

5.Conclusion

The Student Record Management program successfully uses a singly linked list to store student details. It performs insertion, deletion, searching, and display operations. This program helps us understand the basic working and practical use of linked lists in C++.

